

best first search treasure search

Name:Asmit Sahu roll:23052231

```
In [1]: import heapq

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def best_first_search(grid, start, goal):
    rows, cols = len(grid), len(grid[0])

    pq = []
    heapq.heappush(pq, (manhattan(start, goal), start))

    visited = set()
    parent = {start: None}

    while pq:
        _, current = heapq.heappop(pq)

        if current == goal:
            break

        if current in visited:
            continue
        visited.add(current)

        x, y = current
        directions = [(1,0), (-1,0), (0,1), (0,-1)]

        for dx, dy in directions:
            nx, ny = x + dx, y + dy

            if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 0:
                neighbor = (nx, ny)
                if neighbor not in visited:
                    parent[neighbor] = current
                    heapq.heappush(
                        pq,
                        (manhattan(neighbor, goal), neighbor)
                    )

    path = []
    node = goal
    while node:
        path.append(node)
        node = parent.get(node)
    path.reverse()
```

```
return path
```

```
In [2]: grid = [  
    [0, 0, 0, 0],  
    [1, 1, 0, 1],  
    [0, 0, 0, 0],  
    [0, 1, 1, 0]  
]  
  
start = (0, 0)  
treasure = (3, 3)  
  
path = best_first_search(grid, start, treasure)  
print("Path to Treasure:", path)
```

Path to Treasure: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (3, 3)]